

WIFI STA Mode Experiment

WIFI STA Mode Experiment

- 1. WIFI STA Mode Overview
- 2. Core Concepts
 - 2.1 How STA Mode Works
 - 2.2 Network Configuration Parameters
 - 2.3 Core API
- 3. Hardware Dependencies
 - 3.1 ESP32-S3 WiFi Hardware Features
- 4. Project Architecture and Flow
- 5. Source Code Explained
 - 5.1 Configuration Parameters
 - 5.2 STA Connection (No Retry Limit, Loop Until Success)
 - 5.3 Printing Network Information
 - 5.4 Main Program Entry
- 6. Operating Procedures
 - 6.1 Flashing and Running
 - 6.2 Expected Results
 - 6.3 Serial Output Example
 - 6.4 Verification Methods
- 7. Common Problems

1. WIFI STA Mode Overview

STA (Station, client mode) is the most common way for the ESP32-S3 to go online — just like a phone connecting to WiFi, it actively connects to an existing wireless router or hotspot, obtains an IP address, and then can access the Internet or other devices on the LAN.

This experiment configures the ESP32-S3 in STA mode; after connecting to a specified WiFi network, it prints the network status information (IP, mask, gateway, DNS, signal strength).

Typical application scenarios:

Application Type	Example
IoT devices	Smart home sensors report data to the cloud
Data acquisition	Temperature/humidity nodes send data to a server via WiFi
OTA upgrade	Download firmware updates from a server
Time synchronization	Connect to an NTP server to obtain network time

In STA mode the ESP32-S3 is a "client" and needs an existing router/hotspot to access. If you need the ESP32-S3 to create its own hotspot, refer to the WIFI AP Mode experiment.

2. Core Concepts

3.1 ESP32-S3 WiFi Hardware Features

Feature	Parameter
WiFi standard	IEEE 802.11 b/g/n
Operating band	2.4 GHz
Max transmit power	+21 dBm (11b)
Max receive sensitivity	-97 dBm (11b)
Supported protocols	WPA/WPA2/WPA3 personal/enterprise
Antenna	On-board ceramic antenna / IPEX external antenna
Operating modes	STA / AP / STA+AP / Sniffer
Channel range	1 ~ 14

4. Project Architecture and Flow

```
Script execution (single-file .py)
|
|- wifi_connect(): connect in a loop with no retry limit; print IP after
connecting
|- wifi_info(): print IP/mask/gateway/DNS/signal strength
|
|- main():
    |- connect to WiFi (while True until connected)
    |- print network info (IP/mask/gateway/DNS/RSSI)
```

5. Source Code Explained

Full source code: [Source Code](#) -> [MicroPython](#) -> [4.Network Protocols](#) -> [WiFi_STA](#) -> [WiFi_STA.py](#)

5.1 Configuration Parameters

The SSID and password can be modified according to the actual environment; `RETRY_WAIT` controls the retry interval after a connection failure.

```
WIFI_SSID = "Your Wi-Fi SSID"      # Wi-Fi SSID
WIFI_PASS = "Your Wi-Fi password"  # Wi-Fi password
RETRY_WAIT = 3                     # retry interval (s)
```

5.2 STA Connection (No Retry Limit, Loop Until Success)

The `while True` loop runs until connected; each round checks the connection for 15s, and on failure it waits `RETRY_WAIT` seconds before retrying, with no limit on the number of attempts.

```
def wifi_connect():
```

```

"""Connect WiFi, loop until connected"""
wlan = network.WLAN(network.STA_IF) # create STA interface

print(f"[STA] Connecting to '{WIFI_SSID}' ...")
while True:
    try:
        wlan.active(False)
        time.sleep_ms(500)
        wlan.active(True)
        time.sleep_ms(200)

        if wlan.isconnected():
            print(f"[STA] Already connected. IP: {wlan.ifconfig()[0]}")
            return wlan

        wlan.connect(WIFI_SSID, WIFI_PASS)
        for _ in range(15): # 15s timeout
            if wlan.isconnected():
                print(f"[STA] Connected! IP: {wlan.ifconfig()[0]}")
                return wlan
            time.sleep(1)
    except OSError as e:
        print(f"[STA] OSError: {e}")
    print(f"[STA] Retrying in {RETRY_WAIT}s...")
    time.sleep(RETRY_WAIT)

```

The retry count is no longer limited — the `while True` loop runs until connected. Each round checks the connection status within 15s, and after failure it waits 3s and retries automatically.

5.3 Printing Network Information

`ifconfig()` returns (IP, subnet mask, gateway, DNS), and `status('rssi')` returns the signal strength in dBm.

```

def wifi_info(wlan):
    cfg = wlan.ifconfig()
    print(f" IP      : {cfg[0]}")
    print(f" Mask    : {cfg[1]}")
    print(f" Gateway : {cfg[2]}")
    print(f" DNS     : {cfg[3]}")
    print(f" RSSI    : {wlan.status('rssi')} dBm")

```

`ifconfig()` returns a (IP, subnet mask, gateway, DNS) quadruple, and `status('rssi')` returns the signal strength; the closer to 0, the stronger the signal (e.g. -40 dBm is good, -80 dBm is weak).

5.4 Main Program Entry

Calls `wifi_connect()` which blocks until connected, then prints the IP/mask/gateway/DNS and signal strength.

```

def main():
    wlan = wifi_connect() # connect to WiFi
    wifi_info(wlan)       # print network info

```

6. Operating Procedures

6.1 Flashing and Running

Thonny IDE steps: See `MicroPython -> 1. Before Development -> 3. Thonny IDE`.

1. Modify the `WIFI_SSID` and `WIFI_PASS` in the code to an actually available WiFi network
2. Open `wifi_sta.py` in Thonny IDE and click Run

6.2 Expected Results

After a successful connection, the IP address and network parameters are printed; you can `ping` the IP to verify network reachability.

6.3 Serial Output Example

```
[STA] Already connected. IP: 192.168.11.229
IP      : 192.168.11.229
Mask    : 255.255.255.0
Gateway : 192.168.11.1
DNS     : 192.168.11.1
RSSI    : -9 dBm
```

```
[STA] Already connected. IP: 192.168.11.229
IP      : 192.168.11.229
Mask    : 255.255.255.0
Gateway : 192.168.11.1
DNS     : 192.168.11.1
RSSI    : -9 dBm
```

6.4 Verification Methods

Operation	Expected Result
Enter correct SSID/password	Connection succeeds, IP is printed
Enter a wrong password	Retries every 3s until connected
After successful connection, <code>ping <IP></code> (on PC)	Can ping the ESP32-S3

7. Common Problems

Symptom	Cause	Solution
Keeps printing retries	Wrong SSID/password or too weak signal	Confirm the WiFi is 2.4 GHz (the ESP32-S3 does not support 5 GHz), check the password
IP obtained as 0.0.0.0	DHCP did not assign an IP	Check whether the router's DHCP service is working properly

